天猫AI助手：调度框架重构与AI Coding工程化实践

原创 天猫技术团队 大淘宝技术 2026年8月11日 19:12 浙江 10人

天猫AI助手团队通过“调度框架重构”与“AI Coding工程化”双轮驱动，在两周内将底层架构从硬编码升级为基于Reducer/Event的三层调度模型，实现了状态写入收敛、可观测性独立通道及业务接入效率的大幅提升；同时建立包含范式、Skill、Hook及观测报告的AI工程化闭环，将单业务接入成本从3.5−4.5人天降至约0.7人天，验证了将隐性知识显式化为组织工程能力对于应对业务泛化迭代及穿越模型代际更迭的核心价值。

01 前情提要

我们旧瓶装新酒，对天猫AI助手的调度框架做了一次较彻底的改造，同时把团队的 AI Coding 从"个人用 Agent 写代码"升级为"组织化的工程能力"。两条线交叉推进，最终达成几个可量化结果：

- **业务承载力**：从单一讲价流程，扩展到 5+ ScheduleBizType / 5+ ProcessTemplate 共存，新增业务接入压平到 < 1 天；
- **状态写复杂度**：21 处散点 `query + CAS + retry` 收敛到 2 个 Reducer + 25 个 Event，单 process 终态写库次数从 4 次降到 2 次；
- **可观测性**：从一份 `assistShared.log` ，拆成 monitor / bizStat / error 三通道独立采样、独立降级；
- **AI Coding 工程化**：4 份范式 + 13 个 Skill + 8 个 Hook + 月度观测报告形成闭环，单业务接入从 3.5–4.5 人/天压到 ~0.7 人/天。

本文不是论文，是一次复盘，重点想分享三件事：

1. 状态写收敛这件事到底怎么落、为什么收益是非线性的；
2. **AI Coding** 在团队层面到底应该做成什么样，我们走的路子和数据；
3. 中间踩过的坑，希望对其他团队有借鉴价值。

02 背景：业务快速迭代与泛化探索的诉求

业务泛化探索对底层框架提出了两个直接诉求：

诉求 1：业务能"快速迭代"。 一个新业务从立项到上线的周期，不能再是"先排几周框架对齐、再写业务逻辑"。理想状态是业务方拿到清晰的接入范式后，专注写自己的业务节点，框架层面的状态管理、调度推进、可观测性这些事，不应该让业务侧重新发明一遍。

诉求 2：框架能"承载多形态"。 不同业务的核心节点形态差异很大，但底层共享调度、状态机、可观测性、灰度等能力。框架需要"既支持已知形态，也不堵死未知形态"。

回到调度框架自身的现状，原来的实现是节点链硬编码在 `process/v1/` ，节点状态靠手写 if-else 推进。这种实现在单一场景下跑了一年多没问题，但放到泛化探索的语境里，几个结构性约束就开始显现：

- **状态写入散点化**：service 模块里能数出 21 处对 `scheduleBO` / `processBO` 的写入，每处自带 `query → mutate → CAS → retry` 。每接一个业务就要复刻一遍这套模板，且并发场景下不同组件之间的写入容易互相覆盖；
- **业务边界不清**：新业务接入要么改 v1（侵入老逻辑、引入回归），要么 fork 一份（复制粘贴、长期分裂），缺一个让"业务彼此物理隔离、共享底层框架"的中间层；
- **可观测性靠 grep**：所有日志糊在一份 `assistShared.log` 里，大促降级只能"全开/全关"，SLS 聚合也聚不起来；
- **隐性知识门槛高**：框架的约定散在老员工脑子里——什么时候用 CAS、状态怎么流转、节点结束后谁负责推进——没文档、没工具、没拦截，新业务接入第一周基本在"读老代码 + 问范式"。

这些约束指向的不是"老代码不好"，而是"老代码是为单一场景优化的，不适合泛化"。要让业务能快速迭代、让框架能承载多形态，调度框架本身必须做一次结构性升级。

与此同时，我们还押注了第二件事——**把团队的 AI Coding 从"个人用 AI Coding"升级为"组织化的工程能力"**。这两件事天然耦合：泛化框架需要把"约定"显式化、范式化，而范式恰好是 Skill 的素材；反过来，Skill 加速业务接入口能验证范式是否真的好用。两条线必须一起推。

下面分两条线讲清楚我们做了什么。

03 系统线：从硬编码节点链到三层调度 + Reducer/Event收敛

3.1 整体演进

不是一次性推倒重来，而是分阶段非破坏性演进：

```
v1（历史）          硬编码节点链，仅讲价场景使用
   │
   ▼
pro（过渡期）       抽象 ProcessNode / Executor / Builder / Syncer，按业务
   │
   ▼
framework（本次）    ┌─ DAG 层：依赖/取消/级联收敛
                    ├─ State 层：Reducer + 25 Event 写收敛
                    ├─ Log 层：三通道 + Observer 横切
                    └─ Skill 体系：范式 + Hook + 观测
```

老的 v1 流程仍在跑，不强推迁移；pro 是过渡期产物，新业务一律走 framework。这是关键决策——我们没有为了"架构干净"去强行重写 v1 老代码，因为老代码风险大、收益低，留一份"考古证据"反而比重写划算。

3.2 三层调度模型

整个调度被划分成清晰的三层：

```
Schedule 层  — dispatch / cancel / 终态聚合
     │
     ├─ DAG 层  — 初始化、后继推进、失败级联、取消广播
     │
     └─ Process 层  — ReAct 循环、Node 调度、Syncer 钩子
           │
           └─ Node 层  — 原子动作，返回 NodeResult(CONTINUE/SUSPEND)
```

每一层都有自己的状态机：`ScheduleBizStatus` → `ProcessBizStatus` → `ProcessNodeStatus`。粒度由外到内递减。

这种分层最大的好处是**业务侧只关心 Node 层**——节点是个原子动作，输入是 BO，输出是 `NodeResult(CONTINUE/SUSPEND)`，永远不需要操心 schedule 怎么终结、DAG 怎么传播取消、状态怎么入库。这些事情上层框架统一处理。

3.3 写收敛：本次改造的"压舱石"

这块是改造的核心，也是收益最大的部分。

老问题：

21 处散点写入，每处都长这样：

```
// 老代码风格（散点 CAS）
ProcessBObo= processDaoService.query(processId);
if (bo.getStatus() != EXPECTED_FROM) return;
bo.setStatus(NEW_STATUS);
bo.setSomeField(...);
booleanok= processDaoService.updateByCas(bo, expectedFrom);
if (!ok) {
// retry...重新 query 一次再 CAS
}
```

这种写法的问题，并不是"写得不好"，而是**它把三件本应分开的事情糅在一起**：

1. 前置条件校验（precondition）；
2. 字段变更（mutation）；
3. 并发冲突处理（retry）。

业务侧每次写新组件时，都要重复实现这套模板，且每次写法都略有不同。一旦冲突场景复杂（比如多个组件并发改同一行），CAS 失败的 retry 策略很容易踩坑。

新设计：Reducer + Event。

借鉴了 Redux 的思路，但落到 Java + 数据库的语境下：

```
// 新代码风格
scheduleReducer.dispatch(scheduleId, newMaybeAdvanceToTerminalEvent(...
```

业务侧只表达"我希望发生 X"，至于 X 在什么条件下能发生、和数据库当前状态如何对账、CAS 失败要不要重试——全部由 Reducer 内部统一处理。

收益不是线性而是结构性的：

维度	改造前	改造后
schedule 写点	9 处散点	1 个 ScheduleReducer
process 写点	12 处散点	1 个 ProcessReducer
Event 数	—	25 个事件类型
单 process 终态写库次数	4 次 CAS	2 次 CAS
`processDaoService.update*` 直调	多处	0 处
`scheduleDaoService.update*` 直调	多处	0 处

特别说明一下"单 process 终态写库次数从 4 次降到 2 次"。在 N 个子任务并发推进的场景下，这是 N 倍放大效应——一个含 10 个子任务的主任务，旧框架下终态聚合可能要 40 次写，新框架下只要 20 次。

3.4 25 个 Event 是不是太多了？

第一次看到这个数字时，团队内部也讨论过：是不是过度设计？

我们的判断是：**Event 数不是越少越好，而是要"覆盖所有有意义的状态跃迁"**。25 个 Event 大体可以分成几类：

- **生命周期类**（Initial / Terminal）：每种实体的初始化、终态推进；
- **业务驱动类**（NodeFinish / ProcessSuspend / ProcessResume）：业务节点产生的状态变化；
- **DAG 协同类**（DagAdvance / DagCancelBroadcast / ContextReconcile）：DAG 层依赖/取消/对账；
- **聚合类**（MaybeAdvanceToTerminal / SummarizeAndAdvance）：跨实体的状态汇总。

每个 Event 都对应一个明确的语义。如果硬把语义不同的事件合并，业务侧用起来反而要在一个 Event 里塞各种 if-else 判断模式，反过来污染 Reducer。

写收敛的本质是**把"如何写"和"为什么写"分离**——Reducer 管"如何写"（precondition/mutation/retry），Event 描述"为什么写"。这层分离让 schedule / process 两块状态从此再没出过 P0 级问题。

3.5 一些关键的技术取舍

挑几个对外可借鉴的：

取舍 1：DAG 调度内置在 framework，没用外部调度引擎。原因：业务节点深度依赖 inline ReAct（节点内多轮调用 AI 模型），外部调度引擎的"任务粒度"太粗，且会引入额外的中间件依赖。代价是要自己维护调度逻辑，但收益是灵活、可控、链路短。

取舍 2：取消是"同步闭环"而非"异步广播"。端侧 cancel 用户是立即点的，体验上必须秒回。我们让 cancel 复用 dispatch 锁，做同步 CANCELLING → CANCELLED 闭环。代价是 cancel 和正在执行的节点会有锁竞争，但在我们的场景下节点都是短任务，竞争窗口可控。

取舍 3：日志三通道独立 logger，而不是 MDC 区分。最初想用 MDC 区分日志类型（一个 logger，多个 MDC tag），但 Async appender 下 MDC 传递在跨线程时不可靠，且无法做"通道级"降级。最后决定拆成 3 个独立 logger，配 8 个 SwitchCenter 开关覆盖通道总开 / eventType / bizType / 采样率四个维度。

取舍 4：横切关注点用 Observer 而不是装饰器嵌套。Reducer 的写入路径上有指标埋点、审计、灰度判断等横切逻辑。用装饰器嵌套层数会爆炸，用 Observer 模式让 Spring 自动收集，新增观察者零侵入。代价是 Observer 抛异常需要单独隔离不污染主流程。

3.6 可观测性：让 oncall 有数据可看

三通道日志规约：

- `schedule_monitor.log`：高频行为日志（dispatch / execute / cas / dag.advance / compensation.scan）
- `schedule_biz_stat.log`：业务结果日志（schedule.terminated / process.complete / process.suspend）
- `schedule_error.log`：ERROR + 堆栈（含 reducer.apply.error / notifier.{syncer,observer}.failed 等 7 类）

字段做了白名单约束，eventType 做了字典约束。这两个约束保证 SLS 能直接 group by 出指标，不再依赖 grep。

落到 SLS 看板的关键指标：

- schedule 终态时延 P50/P95/P99（按 bizType）
- reducer CAS 重试率 / exhausted 计数
- 单 process 写库次数（迁移前后对比）
- process.suspend 比例 + payload 类型分布

oncall 从"凭经验 grep"升级到"看大盘做决策"，这一步是把稳定性从 P0/P1 升级到 SLO 治理范畴的前提。

4.1 我们对 AI Coding 三种水位的判断

AI Coding 在团队里的成熟度，可以粗略分成五个水位：

水位	表现	主要风险
L1 个人使用	各人各用各的，无规范	输出质量参差、反模式回流
L2 团队约定	有 prompt 模板 / cheatsheet	约定不被执行，靠自觉
L3 工程化	**范式 → Skill → Hook → 观测的闭环**	**持续维护成本，需 owner**
L4 度量驱动	落地率 / 绕过率 / GAP 指标 SLO 化	数据噪声需清洗
L5 自适应	模型 + 范式协同进化	暂未到

大部分团队卡在 L1 / L2：每个人都在用 Agent，但团队没有沉淀。组里资深员工写的 prompt 不会传给新员工，反模式踩过就忘，下次换个人继续踩。

我们这次想做的事情是：把团队的 AI Coding 推到 L3，并在 L3 稳定后向 L4 过渡。

4.2 工程化飞轮

整个体系是一个有反馈回路的飞轮：

```
设计文档 design/          ──┐
                            │ 抽出
研发范式 paradigm/         ──┐
                            │ 编码进
Skill 矩阵 (.agent/skills/) ──┐
                            │ 触发
agent 执行                  ──┐
                            │ Hook 监听
.agent/hooks/* + logs/     ──┐
                            │ 跑分析
analyze-skill-usage        ──┐
                            │ 反馈
docs/local/skill-usage-report.md ──┐
                            │ 调优
```

```
15      回到 design / paradigm / skill ——┘
```

逐层拆解：

1. **设计文档（design/）**

   存放每个核心模块的"长期决策"，比如《调度框架设计文档》《日志设计方案》《Skill使用观测设计》。这些文档不轻易改，是知识资产。

2. **研发范式（paradigm/）**

   "做 X 这件事的标准流程"，比如《新增主任务研发范式》《新增子任务研发范式》。范式 = 步骤清单 + 反模式列表 + 标准锚点路径。范式是 Skill 的素材，但本身也是给人看的文档。

3. **Skill 矩阵（.agent/skills/）**

   范式被编码成 agent Code Skill，让 agent 在合适的场景下被自动触发，按范式执行。我们沉淀了 13 个 Skill，可以分成几类：

类别	Skill	作用
脚手架	add-schedule-biz-type / add-process-template	一键生成 11 个骨架文件
适配	analyze-grayconfig / grayconfig-multi-platform	@GrayConfig → 天猫 @BizGrayConfig 自动适配
扫描	scan-ab-experiments	扫已推全实验、提示清理
观测	analyze-skill-usage	月度 skill 使用 + GAP 报告
文档	check-docs-sync	PR 前检测 design/paradigm 是否漂移
元	skill-creator	创建/优化其他 skill
业务	requirement-analysis / generate-test-doc / ...	提测文档、需求分析等

4. **Hook 数据采集（.agent/hooks/）**

Skill 是被触发的、prompt 是被发出的、commit 是落地的——这中间发生了什么我们不知道。Hook 弥补这个缺口：

- `SessionStart` / `UserPromptSubmit` → 记录所有 prompt
- `PreToolUse` / `PostToolUse` → 记录 Skill 调用、工具调用
- `Stop` → 记录每次 session 结束、打 commit 快照
- `PreCompact` → 记录上下文压缩

8 个 Hook 落到 `.agent/logs/` 下四类日志：skill-usage / prompts / commit-snapshot / edits。Hook 这层是"数据基础设施"，没它就没有后面所有的度量。

5. **观测分析（analyze-skill-usage）**

每月跑一次，输出 6 类报告：

- §0 数据来源（按 hostname 聚合）
- §1 使用频率（哪些 skill 真在用）
- §2 意图触发（哪些 prompt 没触发到 skill）
- §3 落地率（skill 启动后 6h 内是否产生 commit）
- §4 绕过率（commit 改了 skill 锚定路径但没启动 skill）
- §5 GAP（模型输出的 Edit/Write 是否落到 commit）

6. **反馈回路**

报告中暴露的问题反过来调 description / 调范式 / 加 hook，形成闭环。

4.3 落地数据：好的、不好的、灰色的都摊开讲

这是 ATA 复盘里最重要的部分——不光讲成功，把数据真实摊开来。

好的部分：

`add-schedule-biz-type` 和 `add-process-template` 是高频 + 高落地率的"健康 skill"。aiAddOn 接入这次实测，11 个骨架文件 5 分钟生成完毕（人工抄 findDiscount 至少半天），且骨架里 12 个反模式 checklist 帮助 Code Review 提前拦住了 3 个潜在问题。

`check-docs-sync` 在最近一次发版前发现了 docs/design/调度框架设计文档.md 的 §6 章节锚点路径已变更，避免了文档漂移。

不好的部分：

绕过率有 8 条记录，集中在 `findDiscount/node/` 系列。也就是说，开发者改了节点目录下的文件，但没启动相应 skill。原因有两个：

1. 这些 skill 的 description 写得不够"主动"——agent 没意识到要触发；
2. 部分修改是小改（改个常量、加个日志），开发者主观觉得没必要走 skill。

第一个问题是 description 优化的事，可以解；第二个问题反映了一个深层 trade-off——**range 太广 skill 会过度触发，range 太窄又会漏触发**。我们当前的策略是"宁可漏触发也不过度触发"，因为过度触发会让开发者烦，长期反而降低主动使用率。

灰色的部分：

GAP 指标上，最近一份报告显示有 10 个文件 agent Edit 过但 0 落 commit；commit 中又有 8 个文件 0 来自 agent。展开看，原因杂得很：

- agent 改对了但开发者本地手工又改了一版（重写）
- agent 改了 90% 但最后 10% 开发者觉得快而手写
- agent 完全错了，开发者放弃

这部分数据噪声很大，但**它逼我们正视一个现实**：模型输出 ≠ 实际落地，团队真正受益的代码量，需要从 commit 角度反推，不能只看 agent 调用次数。

4.4 ROI 估算：一次新业务接入

抽一个真实 case 做 Fermi 估算（aiAddOn 接入这次为例）：

工作量项	改造前估算	改造后估算	节省
看代码理解既有 findDiscount 模式	1 人/天	0.2 人/天（读范式）	-0.8
创建 11 个骨架文件	0.5 人/天	5 分钟	-0.5
反模式踩坑 + Code Review 来回	1–2 人/天	0.3 人/天	-1.0
文档同步	0.5 人/天	0.1 人/天	-0.4
日志埋点	0.5 人/天	0.1 人/天	-0.4
合计	**3.5–4.5 人/天**	**~0.7 人/天**	**~3 人/天/次**

按业务接入频率估，年度节省 40—80 人/天。考虑到体系本身的维护成本（月度 1—2 人/天），ROI 是正的。

但**我们也想坦诚承认这里的不确定性**：

- 度量本身有噪声（落地率受 prompt 复杂度影响）
- Skill 维护是软成本，团队人员流动会冲击
- 范式过严会反向降低主动使用率

ROI 表上的数字是真实的，但**它代表的是上限，而不是默认值**——前提是范式被遵守、skill 被触发、报告被复盘。

4.5 与上游能力的边界

很多人问过我："你们做这套不是和 agent Code 自己的能力重复吗？"

讲清楚边界很重要：

- **上游（Anthropic / agent Code）能力**：模型本身、tool use、prompt cache、Skill / Hook / Plan Mode / Agent 这些原语、IDE 集成。这些我们不维护，跟着升级。
- **我们的能力**：业务范式、Skill 内容、Hook 业务规则、观测分析、组织约定。这些必须我们维护，没人替我们写。

类比一下：上游给的是"乐高零件"，我们组装的是"乐高城堡"。模型能力会持续涨，但**组织知识资产是真正的壁垒**——模型升级时，我们的范式 + skill 是顺风车。



文章不讲坑就不完整。挑几个值得分享的：

坑 1：DagTerminationChecker 下线那次的"幽灵 CANCELLING"

第一版写收敛时，留了一个 `DagTerminationChecker` 做兜底——觉得"万一 Reducer 没推进到终态，让 checker 来补"。结果上线后偶发出现"主任务永久卡 CANCELLING"。

排查发现，checker 和 reducer 在某些时序下会互相覆盖对方的 mutation。本质是双兜底反而引入了竞争——两个组件都觉得自己是终态聚合的最终决策者。

后来下线了 checker，统一走 `MaybeAdvanceToTerminalEvent + SummarizeAndAdvanceEvent`，问题消失。

教训：收敛要彻底，不要"先收敛 + 再加兜底"。兜底的存在会让收敛失效，因为业务侧总会绕到兜底路径上去。

坑 2：Skill description 太宽 vs 太窄

最早写 `analyze-grayconfig` 时，description 写得偏窄："仅在用户提到 GrayConfig 时触发"。结果发现很多 PR 改了配置但触发不到。改宽后又陷入另一个极端——非配置改动也偶尔被触发。

最后定下的规则是：**description 写"什么场景该触发"+ "什么场景不该触发"两段**，明确边界比堆关键词有效。这条经验后来沉淀成了 `skill-creator` 的内置约束。

坑 3：Hook 数据隐私

Hook 会记录所有 prompt。最早实现时直接写文件、没脱敏。后来内部 Code Review 时被指出可能记录到敏感字段（虽然主要是技术 prompt，但偶尔会带 cookie/token）。最后给 Hook 加了简单的脱敏规则 + 日志路径权限控制。

教训：采集即责任。任何动 telemetry 的项目，第一天就得想清楚数据治理。

坑 4：范式过严反而降低使用率

第一版 `add-schedule-biz-type` 范式列了 16 个反模式，每条都强校验。结果开发者觉得"用 skill 比手写还累"——因为骨架太严格，业务有点点偏离就要绕。

后来调整成"反模式分级"——P0 级（必须拦截，比如 Node 直推 GUI）+ P1 级（提醒不强拦）+ P2 级（仅在 review 时建议）。落地率明显回升。

教训：范式是给人用的，不是给人受罪的。"严"和"好用"是冲突的，需要持续微调。



可对外参考的方法论

如果你的团队也想做类似的事情，下面这些经验大概率有用：

6.1 系统重构方面

1. **非破坏性演进 > 推倒重来。**我们让 v1 / pro / framework 三代并存，新业务一律走最新代，老代码留作"考古证据"。这种打法的关键是给老代码画清楚边界，避免"两边都改"。
2. **写收敛是高 ROI 的重构方向。**状态机散点写、CAS 散点 retry 是大部分成熟系统的通病。Reducer + Event 的范式不需要新中间件，落地成本低，收益持续。
3. **分层不是越多越好，是"每层有清晰职责"。**我们的 Schedule / DAG / Process / Node 四层，每层只负责一类决策。判断分层是否合理的标准：每层的代码能否独立讲清楚。
4. **可观测性投入要前置。**日志通道拆分、字段白名单、SLS 看板，这些在重构同期做最划算——重构本身会大量改写日志点，不如顺手改成新规约。

6.2 AI Coding 方面

1. **范式先于 Skill。**先把"做这件事的标准流程"写成 paradigm 文档，再编码成 Skill。直接写 Skill 容易陷入"对模型说什么"，写范式逼你想清楚"对人说什么"。
2. **Hook 是 telemetry 基础设施，越早做越好。**没有数据就没有反馈回路，整个飞轮转不起来。Hook 最便宜的实现就是"prompt + tool 调用 + commit"三类日志，月度跑一次分析。
3. **Skill 的 description 是 70% 的工作。**写 Skill 内容不难，难的是让 agent 在正确的场景下"想到"用它。description 不是关键词堆叠，是边界声明——什么时候用 / 什么时候不用都要明说。

4. **观测报告要敢于摊开数据**。落地率低、绕过率高、GAP 大——这些数据比"用了多少次 Skill"更有价值，因为它们暴露问题。月度复盘的 90% 价值在"不健康"的指标里。
5. **维护成本是真实的**。Skill / Hook / 报告需要 owner。我们当前由 1 人兼职可承担，但要清楚：**没有 owner，飞轮一定会停**。

6.3 双轮驱动的本质

回头看，**系统重构和 AI Coding 工程化是同一件事的两面**：

- 重构需要把"老员工脑子里的隐性知识"变成"可执行的范式"；
- AI Coding 工程化需要"可执行的范式"作为 Skill 的素材；
- Skill 反过来加速业务接入，验证范式是否真的可执行；
- 业务接入产生的数据反向调优范式和重构。

如果只做重构不做 AI 工程化，范式会停留在文档上，新员工还是要 1 周看代码；如果只做 AI 工程化不做重构，Skill 编码的是"老代码的写法"，沉淀的是反模式。

两条线必须一起推。



诚实地讲，当前这套体系仍有几个边界：

- **WAITING 子任务无超时机制**。长链阻塞需要主任务级 deadline 兜底，目前还没补；
- **summarize 失败重试无上限**。外部依赖长期不可用时会无限重试，准备加 retryCount 上限；
- **`completedProcessIds` 等集合规模上限**。单主任务 >100 子任务时需评估 JSON 体积，目前业务还没到这个量；
- **Skill 触发率和开发者主观感受耦合**。需要定期 1:1 收集主观反馈，单纯看数据不够；
- **跨仓库 telemetry 聚合**。当前 hook 数据落本地 / 团队内 telemetry repo，跨组复制时需要平台化考虑。

下一步的路线大致：

- 短期（3 个月内）：稳住 L3 体系，月度 skill 报告固化；补 WAITING 超时和 summarize 上限。
- 中期（半年内）：推到 L4——落地率 / 绕过率 SLO 化，反向自动调优 Skill description。
- 中期：跨组复制 POC，验证范式 + skill 体系是否可被搬运（可能需要平台层抽象）。

- 长期：根据模型能力进展，评估是否能做"自动 Skill 生成 / 自动 PR 评审"这类 L5 能力。



写在最后

这次改造的最大体会：**AI Coding 不是个人生产力工具，是团队工程能力。**

个人用 agent 写代码这件事，不需要任何工程化——打开 IDE 装个插件就完事。但当一个团队要让"AI Coding 的收益可度量、可复制、可传承"时，就必须做工程化：把范式写下来、把范式编码成 Skill、把 Skill 的使用观测起来、把观测数据反馈回去。

这套打法不复杂，但需要持续投入。比起买更贵的模型，**让团队的隐性知识沉淀下来**，才是穿越模型代际更迭的核心壁垒。

如果你的团队也在思考"AI 怎么真的帮到我们而不是只帮到个人"，希望本文有点用。欢迎交流拍砖。



团队介绍

本文作者依凡，来自淘天集团—天猫APP技术团队。作为淘天集团的核心技术引擎，我们负责天猫APP的全栈开发与体验升级，支撑导购、搜推、商详、购物车、下单等核心交易链路。我们以极致的端侧性能与创新的Agent架构，打造「AI导购」「AI助手」等面向C端的智能应用，并探索端外场景合作，为千万级用户构建沉浸式、个性化的购物体验，持续引领电商APP的技术演进。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法



大淘宝技术
大淘宝技术官方账号
966篇原创内容

公众号

阅读 5487　修改于2026年8月11日

留言

写留言

还没有留言，来写首评